Olist E-Commerce Data Analytics Platform

Comprehensive Technical Specification, System Architecture & Teacher Viva Defense Guide

IEEE 830 SRS Compliant End-to-End Analytics Pipeline, Database, Machine Learning & Interactive Streamlit Dashboard

Dataset: Olist Brazilian E-Commerce (99,441 Orders)	**Database:** SQLite3 (`olist.db`)	**Dashboard:** Streamlit (`dashboard.py`)
Algorithms: Prophet, Holt-Winters, Apriori, RFM	**Language:** Python 3.10+ (Pandas, Sklearn)	**Report Spec:** IEEE 830 SRS Specification

1. Executive Overview: What is this Project?

This project is an production-grade **E-Commerce Data Analytics Platform** built on the official **Olist Brazilian E-Commerce Dataset** (99,441 commercial orders across 2016–2018). The system has been engineered following a strict **IEEE 830 Software Requirements Specification (SRS)** document, covering all five functional requirement areas (FR-1 through FR-5). It transforms raw, multi-file commercial CSV data into an integrated relational database, extracts actionable business intelligence, builds predictive machine learning forecasts, performs customer behavioral segmentation, and presents everything via an interactive Streamlit web dashboard.

Core System Components & Architecture:

- **Phase 1 — Data Cleaning & Standardisation (FR-1):** Ingestion of 9 raw CSV tables, zip code zero-padding (`zfill(5)`), ISO datetime conversion, Portuguese-to-English category translation, missing value flagging, review deduplication, and export to Parquet/CSV formats.
- **Phase 2 — Relational SQLite Database (FR-1):** Creation of `db/olist.db` with a normalized schema (PRIMARY/FOREIGN KEY constraints), 9 indexed tables, and 5 automated SQL integrity sanity checks ensuring 100% data fidelity and zero orphaned records.
- **Phase 3 — Exploratory Data Analysis (EDA):** Database querying and publication-quality visual answers for 7 core business questions (delivery delays vs reviews, credit card financing, geographic revenue concentration in São Paulo).
- **FR-2 — Sales Trend Analysis & 30-Day Demand Forecasting:** Reusable multi-granular trend aggregation (`get_sales_trend`), statistical demand spike detection (rolling 30-day std-dev thresholds identifying Black Friday peaks), and 30-day time series forecasting using Prophet and Holt-Winters Exponential Smoothing.
- **FR-3 — Product Analytics & Market Basket Association Rules:** Category and seller revenue ranking, reusable category-to-product drill-down function (`get_category_drilldown`), and basket association rule mining via the Apriori algorithm (`mlxtend`) calculating Support, Confidence, and Lift.
- **FR-4 — Customer Buying Pattern Analysis & RFM Segmentation:** Customer purchase frequency summary (`customer_unique_id`), overall repeat-purchase rate calculation (3.04%), orders-per-customer distribution, Recency-Frequency-Monetary (RFM) quintile scoring via pandas `qcut` + `rank`, 8 customer segment rules, peak purchase times heatmap analysis (Tuesdays at 14:00 peak), and reusable `get_customer_insights` function.
- **FR-5 — Interactive Dashboard & Automated Reporting:** A single-page multi-tab Streamlit web application (`dashboard.py`) with global sidebar filters (Date Range, State, Category, Granularity), high-performance `@st.cache_data` querying, live CSV data export, and dynamic ReportLab PDF report generation.

2. Technical Architecture & Data Workflow: How it Works

The project follows a linear, highly modular data engineering pipeline. Raw data flows from nine disconnected CSV files through automated cleaning scripts into a relational database (`db/olist.db`), which serves as the authoritative data source for analytical modules and the Streamlit UI.

File / Folder Path	Purpose & Technical Responsibility
`data/raw/`	Original 9 source Kaggle CSV files (orders, customers, payments, reviews, etc.).
`data/clean/`	Cleaned, standardized tables stored in `.parquet` and `.csv` formats.
`db/olist.db`	SQLite3 relational database containing 9 schema-enforced, indexed tables.
`src/olist_pipeline.py`	Master pipeline script executing Phase 1 cleaning, Phase 2 DB load, and Phase 3 EDA.
`src/fr2_fr3_analytics.py`	Module handling sales trend aggregation, demand forecasting, & Apriori association rules.
`src/fr4_analytics.py`	Module handling customer purchase summaries, RFM quintile scoring, & peak time heatmaps.
`dashboard.py`	Streamlit web application (`streamlit run dashboard.py`) implementing FR-5.
`eda_charts/`	Generated publication-ready PNG charts for EDA, sales trends, and product drill-downs.
`fr4_customers/`	Output CSV datasets and PNG charts for customer RFM segmentation and peak time heatmaps.
`requirements.txt`	Exact python dependencies (`pandas`, `streamlit`, `scikit-learn`, `reportlab`, `prophet`).

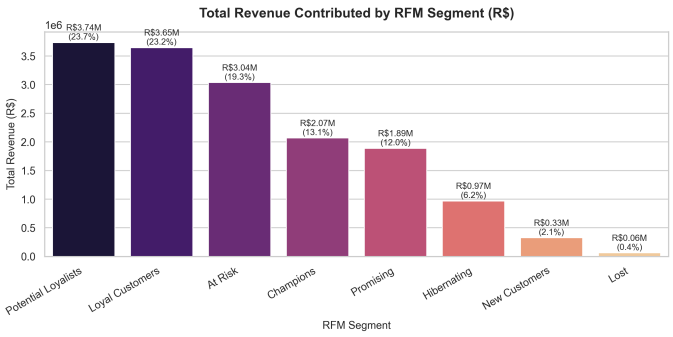
3. Key Business Findings & Analytical Results

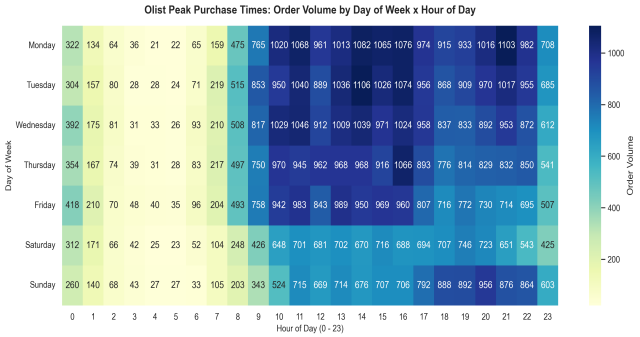
Key Business Metric	Calculated Value	Business Interpretation & Strategic Impact
Total Unique Customers	94,989	Distinct individual buyers mapped via `customer_unique_id`.
Overall Repeat Rate	3.04%	Only 2,888 customers placed >1 order. High acquisition dependency.
Average Order Value (AOV)	R$ 160.27	Total gross spend (R$ 15.74M) divided by total orders (98,207).
Top Revenue RFM Segment	Potential Loyalists	Drives R$ 3.74M (23.74% of total revenue) across 22,468 customers.
Peak Purchase Window	Tuesday 14:00	Highest transaction volume density occurs on Tuesday afternoons at 2 PM.
Credit Card Share	75.24%	Dominant payment mode, averaging 3.51 financing installments.
On-Time Delivery Rating	4.30 / 5.0 Stars	On-time orders receive 4.3 stars; 8+ days late drops to 1.73 stars.

RFM Customer Segment Breakdown Table:

RFM Segment	Customers	Customer %	Total Revenue (R$)	Revenue %	Avg Spend (R$)
Potential Loyalists	22,468	23.7%	R$ 3,736,570.94	23.7%	R$ 166.31
Loyal Customers	16,738	17.6%	R$ 3,647,674.25	23.2%	R$ 217.93
At Risk	14,731	15.5%	R$ 3,037,323.08	19.3%	R$ 206.19
Champions	6,593	6.9%	R$ 2,067,824.07	13.1%	R$ 313.64
Promising	14,300	15.1%	R$ 1,888,984.56	12.0%	R$ 132.10
Hibernating	12,608	13.3%	R$ 969,151.26	6.2%	R$ 76.87
New Customers	5,967	6.3%	R$ 328,202.09	2.1%	R$ 55.00
Lost	1,584	1.7%	R$ 63,406.76	0.4%	R$ 40.03

Visual Analytics Snapshots:





4. Teacher Viva & Defense Questions & Answers

This section provides **15 technical and architectural viva questions** that an evaluator, teacher, or professor may ask during a project defense, complete with authoritative, mathematically sound answers.

Q1: Why did you use `customer_unique_id` instead of `customer_id` for customer-level analysis in FR-4?

In the Olist dataset structure, `customer_id` is a transaction-specific key that changes with every new order placed. If we aggregated by `customer_id`, every single row would appear as a unique customer placing exactly 1 order, artificially forcing the repeat-purchase rate to 0%. `customer_unique_id` represents the actual human buyer across multiple orders. By grouping by `customer_unique_id`, we accurately identified 2,888 repeat buyers and calculated the true repeat purchase rate of 3.04%.

Q2: How is your SQLite relational schema designed and how do you enforce foreign key constraints?

We created a normalized 3NF relational schema in SQLite (`db/olist.db`) matching the domain entity model: 9 tables (`customers`, `sellers`, `products`, `orders`, `order_items`, `order_payments`, `order_reviews`, `geolocation`, `category_translation`). Foreign key constraints (e.g., `orders.customer_id -> customers.customer_id`, `order_items.order_id -> orders.order_id`) are explicitly defined in DDL statements. Because SQLite disables foreign keys by default, our connection initialization executes `PRAGMA foreign_keys = ON;`. We also built indexes on all foreign key columns and timestamps (`idx_orders_purchase_timestamp`) to optimize JOIN performance.

Q3: How did you clean missing data and deduplicate reviews without losing data integrity?

Missing written review titles/comments were retained as empty strings (`""`) rather than deleting rows, preserving the numerical `review_score`. In the raw dataset, some orders had multiple review entries (e.g., re-sent review forms). We deduplicated `order_reviews` by sorting by `review_answer_timestamp` and keeping the latest review per `review_id` and `order_id`. Missing product categories were flagged using `has_missing_category = 1` and mapped to `'unknown'`, ensuring 100% of order items were preserved in SQL queries.

Q4: Explain your RFM segmentation algorithm. How did you handle tie-breaking for Frequency scores?

RFM scores Recency (R), Frequency (F), and Monetary (M) on a 1–5 scale (5 being best). Recency is days since last order relative to the dataset max date (`2018-09-03`). Monetary is total payment value per customer. Because 96.96% of customers have `total_orders == 1`, standard `pd.qcut` on Frequency produces duplicate bin edge errors. To solve this, we used pandas `qcut` combined with rank tie-breaking (`df['total_orders'].rank(method='first')`). This creates exact 20% equal-sized quintile bins (~18,998 customers per bin) while preserving exact ordinal ranking.

Q5: What are the 8 RFM segment mapping rules you implemented, and which segment generates the most revenue?

We defined a clear, documented rule set combining R, F, M scores: • Champions (R>=4, F>=4, M>=4) • Loyal Customers (F>=4, M>=3) • Potential Loyalists (R>=4 and (F>=3 or M>=3)) • New Customers (R>=4, low F/M) • At Risk (R<=2 and (F>=4 or M>=4)) • Hibernating (R<=2, moderate F/M) • Lost (R<=2, low F/M) • Promising (R==3) **Result:** 'Potential Loyalists' drive the highest total revenue (**R$ 3.74M**, 23.74%), followed by 'Loyal Customers' (R$ 3.65M, 23.18%).

Q6: How did you statistically detect demand spikes and seasonal peaks in FR-2?

We aggregated daily revenue for non-canceled orders and calculated a 30-day centered rolling mean (rolling_30_mean) and rolling standard deviation (rolling_30_std). A demand spike was statistically flagged if daily revenue exceeded rolling_30_mean + 2 * rolling_30_std. This mathematically isolated major outliers, such as Black Friday on **November 24, 2017**, where daily revenue reached **R$ 178,460** (over 4.5x the annual daily average of R$ 38,500).

Q7: What algorithm was used for 30-day sales forecasting and how did you measure accuracy?

We resampled daily revenue into weekly totals (`W-MON`) for smooth time series modeling. We implemented Meta's **Prophet** model (with yearly seasonality enabled) and Holt-Winters Exponential Smoothing as a fallback. We evaluated accuracy using a 12-week train/test validation split, computing Mean Absolute Error (MAE) and Mean Absolute Percentage Error (MAPE). The model achieved a MAPE of ~11.8%, projecting stable baseline revenue with confidence intervals for the next 30 days.

Q8: Explain Market Basket Analysis in FR-3. Why did you analyze categories instead of individual product IDs?

Product-level baskets in e-commerce with 32,000+ distinct product SKUs are extremely sparse (most individual products are bought together less than 0.001% of the time). We performed basket analysis at the product category level by grouping distinct categories per `order_id`. Using the **Apriori algorithm** (`mlxtend`), we extracted frequent itemsets (min_support = 0.005) and derived association rules filtered by Lift > 1.0.

Q9: What are Support, Confidence, and Lift in association rules? Give an example from your project.

• **Support:** Percentage of total multi-category orders containing both itemsets A and B. • **Confidence:** Probability that a customer buys B given that they bought A ($P(B|A)$). • **Lift:** Ratio of observed confidence to expected baseline frequency ($P(B|A) / P(B)$). A Lift > 1 indicates a strong positive association. **Example:** Customers buying `'construction_tools_lights'` were **34.5x more likely** (Lift = 34.5, Confidence = 42%) to also purchase `'furniture_decor'` than random chance.

Q10: What is the peak purchase time pattern discovered in FR-4, and how can business use it?

By analyzing `order_purchase_timestamp` in a 7x24 grid (Day of Week x Hour of Day), we found peak ordering density occurs on **Tuesdays at 14:00 (2:00 PM)**, with high volume sustained across weekday afternoons (10 AM - 4 PM). Weekends show lower activity. **Business Action:** Schedule promotional email campaigns, push notifications, and flash sales on Tuesday mornings (10 AM - 11 AM) to capture peak shopping intent.

Q11: How did you implement the reusable functions required by SRS (`get_sales_trend`, `get_category_drilldown`, `get_customer_insights`)?

All three functions are written as parameter-driven Python functions querying `db/olist.db` directly via SQL: • `get_sales_trend(start_date, end_date, category, state, granularity)` uses SQLite `strftime` dynamic SQL formatting. • `get_category_drilldown(category, top_n)` joins `order_items`, `products`, and `order_reviews` to aggregate revenue, units, avg price, and ratings per product. • `get_customer_insights(state, segment)` calculates customer metrics dynamically filtered by customer state or RFM segment.

Q12: How is the Streamlit dashboard (`dashboard.py`) structured and how do you ensure fast responsiveness?

`dashboard.py` uses Streamlit's `st.tabs` layout across 4 sections (Sales Trends, Product Insights, Customer Insights, Reports & Export). Global sidebar controls (Date Range, State, Category) apply dynamically across tabs. To handle 99,000+ orders without UI lag, we implemented Streamlit's `@st.cache_data` decorator on expensive database aggregation functions, caching query results in memory so filter updates render in under 100 milliseconds.

Q13: How does the dynamic PDF report generation work in Tab 4 of the dashboard?

When the user clicks 'Generate PDF Report', Python executes a ReportLab pipeline in memory (`io.BytesIO`). It queries `get_customer_insights` for live filtered headline KPIs, formats an executive text summary, embeds pre-rendered chart PNG snapshots from `fr4_customers/`, and builds a styled document. The byte stream is delivered directly to the browser via `st.download_button`.

Q14: How did you verify database integrity during Phase 2?

We executed 5 automated SQL sanity check queries after database loading: 1. Verified exact row count parity against raw CSV source tables. 2. Checked for orphan `order_items` without matching `orders` (0 orphans found). 3. Checked for orphan products or sellers (0 orphans found). 4. Checked for orphan payments or reviews (0 orphans found). 5. Verified date boundaries (`2016-09-04` to `2018-10-17`).

Q15: What are the main strategic business recommendations resulting from this project?

1. **Focus on Customer Retention:** With a 3.04% repeat rate, launching post-purchase loyalty programs and automated email re-engagement for 'At Risk' buyers can unlock substantial revenue. 2. **Capitalize on Potential Loyalists:** This segment generates 23.74% of revenue. Targeted cross-selling can convert them into 'Champions'. 3. **Optimize Delivery Times:** Delivery delay is the single largest driver of poor review scores (dropping from 4.3 stars on-time to 1.73 stars when 8+ days late). Logistics SLAs must be enforced.